



RAG QUANTITATIVE RESEARCH ENGINE **untuk IDX (Indonesia Stock Exchange)**

Retrieval-Augmented Generation untuk Analisis Laporan Keuangan

Disusun Oleh:

Ahmad Naufal Luthfan Marzuqi (Humkom)

Syahrial Nur Faturrahman (Humkom)

Link github :

<https://github.com/Naufallm/RAG-Quantitative-Research-Engine-for-IDX>

Link Colab :

<https://colab.research.google.com/drive/188ifkKA2-moCdbGgDPOEreCLmM0AE1MJ?usp=sharing>

Link Streamlit : <https://idx-ai.streamlit.app/>

1. Perkenalan Materi

Proyek ini membangun sebuah sistem Retrieval-Augmented Generation (RAG) yang dirancang khusus untuk menganalisis laporan keuangan perusahaan-perusahaan yang terdaftar di Bursa Efek Indonesia (IDX).

1.1 Apa itu RAG?

Retrieval-Augmented Generation (RAG) adalah arsitektur AI yang menggabungkan dua komponen utama: sistem pencarian (retrieval) dan model generatif (generation). Konsep ini pertama kali diperkenalkan oleh Lewis et al. (2020).

Cara kerja RAG dapat dijelaskan dalam tiga langkah sederhana:

1. Retrieval — ketika pengguna mengajukan pertanyaan, sistem mencari dokumen paling relevan dari basis data yang sudah diproses sebelumnya
2. Augmentation — dokumen yang ditemukan digabungkan dengan pertanyaan asli menjadi satu konteks lengkap
3. Generation — konteks tersebut dikirim ke LLM untuk menghasilkan jawaban yang akurat

Keunggulan utama RAG adalah kemampuan melacak sumber informasi. Setiap jawaban yang dihasilkan bisa diverifikasi kembali ke dokumen asli, sehingga

meminimalkan risiko informasi palsu (hallucination) yang sering terjadi pada LLM murni.

Bayangkan seorang analis keuangan yang harus membaca ribuan halaman laporan keuangan dari ratusan perusahaan. Dengan RAG, komputer dapat membaca semua laporan tersebut, menyimpan informasinya, dan menjawab pertanyaan spesifik seperti "Berapa laba bersih BBKA tahun 2023?" dengan merujuk langsung ke dokumen asli. Inilah yang dibangun dalam proyek ini.

1.1 Masalah yang Diselesaikan

Analis dan investor menghadapi kesulitan dalam menganalisis laporan keuangan karena:

- Volume data sangat besar — ratusan perusahaan dengan laporan periodik
- Informasi tersebar di banyak file PDF yang tidak terstruktur
- Dibutuhkan keahlian khusus untuk mengekstrak dan menganalisis data keuangan
- Rentan terhadap human error dan butuh waktu lama

1.2 Teknologi yang Digunakan

Proyek ini menggunakan beberapa teknologi utama:

- Python — bahasa pemrograman utama
- LangChain — framework untuk pipeline RAG
- Groq API — akses ke model Llama 3.3 70B (LLM)
- HuggingFace Embeddings — untuk mengubah teks menjadi vektor
- ChromaDB — vector database untuk penyimpanan dan pencarian
- pypdf — ekstraksi teks dari file PDF
- Streamlit & ipywidgets — antarmuka pengguna

2. Penjelasan Pipeline RAG

Pipeline RAG dalam proyek ini terdiri dari enam tahap utama yang berjalan secara berurutan, dari data mentah hingga jawaban analisis keuangan. Berikut adalah penjelasan detail setiap tahap beserta kode yang digunakan.

2.1 Ekstraksi Teks dari PDF

Tahap pertama adalah mengambil teks dari file PDF laporan keuangan. Library `pypdf` digunakan untuk membaca setiap halaman PDF dan mengubahnya menjadi teks string. Nama perusahaan (ticker) diekstrak dari nama file untuk digunakan sebagai metadata.

```
from pypdf import PdfReader

all_raw_documents = []
for filename in pdf_files:
    reader = PdfReader(os.path.join(dataset_path, filename))
    full_text = ""
    for page in reader.pages:
        full_text += (page.extract_text() or "") + "\n"
    ticker = filename.split("-")[-1].replace(".pdf", "")
    all_raw_documents.append({"ticker": ticker, "text": full_text})
```

2.2 Pemotongan Teks (Chunking)

Dokumen keuangan biasanya sangat panjang. Kita perlu memotongnya menjadi bagian-bagian kecil (chunks) agar bisa diproses oleh model. `RecursiveCharacterTextSplitter` memotong teks secara cerdas — pertama pada batas paragraf, lalu kalimat, lalu spasi. Parameter `chunk_size=1000` karakter dengan overlap 200 karakter menjaga konteks antar potongan.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
final_chunks = []
for doc in all_raw_documents:
    for chunk in splitter.split_text(doc["text"]):
        final_chunks.append({"ticker": doc["ticker"], "content": chunk})

json.dump(final_chunks, open("final_chunks.json", "w"))
```

2.3 Vector Database & Embedding

Setiap chunk teks diubah menjadi vektor numerik (embedding) menggunakan model `paraphrase-multilingual-MiniLM-L12-v2` dari HuggingFace. Vektor-vektor ini disimpan di ChromaDB, sebuah vector database yang mampu mencari dokumen berdasarkan kemiripan makna (semantic similarity), bukan sekadar kata kunci. Metrik yang digunakan adalah cosine similarity.

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma

embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
)
vector_db = Chroma(
    persist_directory="chroma_db_idx",
    embedding_function=embedding_model
)
```

2.4 Retrieval (Pencarian)

Ketika pengguna mengajukan pertanyaan, sistem akan:

1. Mengubah pertanyaan menjadi vektor embeddin
2. Mencari 4 chunk teks paling relevan dari ChromaDB (dibatasi pada ticker tertentu)
3. Menggabungkan chunk-chunk tersebut sebagai konteks untuk LLM

Filter ticker memastikan sistem hanya mencari data dari perusahaan yang dimaksud.

```
def get_relevant_docs(query, ticker):
    results = vector_db.similarity_search(query, k=4, filter={"ticker": ticker})
    return results
```

2.5 Integrasi LLM (Groq + Llama 3)

Sistem terhubung ke Groq API yang menyediakan model Llama 3.3 70B. Dengan temperature=0, model akan memberikan jawaban yang konsisten dan faktual. Prompt engineering digunakan untuk membentuk AI sebagai Senior Financial Analyst yang

hanya menjawab berdasarkan data yang diberikan — ini adalah mekanisme anti-halusinasi.

```
from langchain_groq import ChatGroq
os.environ["GROQ_API_KEY"] = "<YOUR_GROQ_API_KEY>"
llm = ChatGroq(model_name="llama-3.3-70b-versatile", temperature=0)
```

2.6 RAG Chain & Output

Semua komponen dirangkai menjadi satu pipeline menggunakan LangChain Expression Language (LCEL). RAG chain ini menghubungkan retrieval → prompt template → LLM → output parser, sehingga satu panggilan fungsi bisa menghasilkan analisis keuangan yang lengkap.

```
rag_chain = (
    {"context": RunnablePassthrough(), "question": RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)

def run_idx_research(ticker, query):
    docs = get_relevant_docs(query, ticker)
    context = "\n\n".join([d.page_content for d in docs])
    return rag_chain.invoke({"context": context, "question": query})
```

3. Bagaimana Proyek Ini Selesai

Proyek ini dikerjakan secara bertahap dengan pendekatan iteratif menggunakan Google Colab sebagai lingkungan pengembangan utama. Berikut adalah alur pengerjaan dari awal hingga selesai:

3.1 Tahap Persiapan Data

Langkah pertama adalah menyiapkan lingkungan pengembangan dengan menginstal library yang dibutuhkan seperti pypdf, langchain, dan pandas. Kemudian dataset laporan keuangan perusahaan IDX dalam format PDF di-clone dari repository GitHub (github.com/Naufallm/RAG-Quantitative-Research-Engine-for-IDX).

Setelah proses clone selesai, memverifikasi jumlah dan integritas file PDF di folder dataset_idx. Verifikasi ini penting untuk memastikan tidak ada file yang corrupt sebelum masuk ke tahap pemrosesan.

3.2 Tahap Pemrosesan Data

Pada tahap ini, mengekstrak teks dari ribuan file PDF menggunakan pypdf. Hasil ekstraksi kemudian dipotong menjadi potongan-potongan kecil (chunks) menggunakan RecursiveCharacterTextSplitter. Proses ini menghasilkan sekitar 9.700+ chunks dari seluruh dokumen. Chunks disimpan dalam file JSON sebagai checkpoint.

Setelah chunking, mengubah setiap chunk menjadi vektor embedding menggunakan model multilingual MiniLM dan menyimpannya di ChromaDB. Proses embedding awal memakan waktu cukup lama, tetapi setelah selesai, database vektor bisa langsung digunakan tanpa perlu mengulang proses.

3.3 Tahap Integrasi LLM

Setelah vector database siap, selanjutnya mengintegrasikan Groq API dengan model Llama 3.3 70B. Tantangan utama di tahap ini adalah:

- Merancang prompt yang tepat agar AI berperilaku sebagai analis keuangan
- Memastikan sistem hanya menjawab berdasarkan data (hallucination guard)
- Mengatur format output agar mudah dibaca

Lalu juga membangun mekanisme filter ticker agar pencarian terbatas pada perusahaan yang dimaksud pengguna.

3.4 Tahap Pengujian

Pengujian dilakukan dalam dua kategori:

1. Uji Akurasi — mengajukan pertanyaan yang jawabannya sudah diketahui dan membandingkan hasilnya dengan data asli di PDF.
2. Uji Anti-Halusinasi — mengajukan pertanyaan yang tidak ada jawabannya di database. Sistem dinyatakan lulus jika menjawab "tidak tersedia" bukan mengarang jawaban.

Kedua pengujian ini penting untuk memastikan sistem dapat dipercaya dalam konteks analisis keuangan.

3.5 Tahap Pembuatan Antarmuka

Dua jenis antarmuka dibangun:

1. Antarmuka Colab (ipywidgets) — untuk penggunaan langsung di notebook dengan form input ticker dan pertanyaan
2. Aplikasi Web (Streamlit) — aplikasi web dengan tampilan modern, sidebar untuk konfigurasi, dan area chat untuk interaksi. File app.py yang telah dibuat bisa dideploy ke Streamlit Cloud agar bisa diakses publik.

3.6 Hasil Akhir

Proyek ini berhasil menghasilkan RAG Quantitative Research Engine yang mampu:

- Memproses ribuan halaman laporan keuangan PDF
- Menjawab pertanyaan analitis secara akurat dan berbasis data
- Menolak memberikan jawaban jika data tidak tersedia (anti-halusinasi)
- Memberikan transparansi dengan menampilkan sumber dokumen

Sistem dapat digunakan oleh analis keuangan, investor, atau mahasiswa yang membutuhkan informasi keuangan perusahaan IDX dengan cepat dan akurat.

3.7 Kesimpulan dan Saran Pengembangan

RAG Quantitative Research Engine ini membuktikan bahwa teknologi RAG dapat diterapkan secara efektif untuk analisis laporan keuangan di pasar modal Indonesia.

Dengan menggabungkan kekuatan pencarian semantik dan generative AI, sistem ini mampu memberikan analisis yang cepat, akurat, dan dapat dipertanggungjawabkan.

Beberapa saran untuk pengembangan ke depan:

1. Data Real-Time — integrasi dengan API IDX untuk mengambil laporan keuangan terbaru secara otomatis, sehingga database selalu up-to-date
2. Multi-Modal RAG — menambahkan kemampuan membaca tabel dan grafik dari PDF menggunakan model vision, karena banyak data keuangan disajikan dalam bentuk tabel
3. Fine-Tuning — melakukan fine-tuning model embedding pada domain keuangan Indonesia untuk meningkatkan akurasi pencarian terminologi keuangan spesifik
4. Deployment Cloud — menyiapkan infrastruktur cloud yang scalable agar sistem dapat diakses oleh banyak pengguna secara simultan
5. Caching — menerapkan mekanisme caching untuk pertanyaan yang sering diajukan agar respons lebih cepat dan mengurangi biaya API.